



DC Motor Control Firmware (STM32)

STM32

CUBEIDE

HARDWARE

IN-THE-LOOP

This repository contains the firmware (C) developed for the real-time control, safety, and data acquisition of a Direct Current (DC) Motor. The STM32 microcontroller acts as the hard real-time core of the system, running a deterministic 100 Hz control loop.

Note: The desktop supervision application (IHM) developed in C++/Qt, which sends commands and receives telemetry from this firmware via UART, is hosted in a separate repository: [ips_mcc_stm32-IHM](#).

☆☆ Main Technical Features

1. Advanced Signal Acquisition & Filtering

- **Speed:** Measured using an incremental encoder read in *Roue Libre* by a hardware timer (TIM3). An external interrupt (EXTI) on Channel I serves as an absolute reference to reset and validate RPM calculations without accumulating errors.
- **Current:** Read via ADC1. To mitigate the 20 kHz PWM switching noise (aliasing), a digital 1st-order IIR Low-Pass Filter (Exponential Moving Average: 30% new, 70% memory) is applied in software before the control loop executes:
$$y[n] = 0.3 \cdot x[n] + 0.7 \cdot y[n-1]$$

2. PWM Generation & Power Drive

- **Hardware Timer:** Driven by **TIM1 (Channel 1)**, configured as an advanced PWM timer.
- **Frequency Configuration:** The timer is configured with a **Prescaler of 41** and a **Period (ARR) of 99**. Running on the system clock, this configuration yields an exact switching frequency of **20 kHz**. This high frequency ensures high electrical efficiency and eliminates acoustic noise, making the chopper operation completely inaudible.
- **Duty Cycle Control:** The resolution ranges precisely from 0 to 99. The duty cycle is adjusted dynamically in hardware by updating the **CCR1** register via `__HAL_TIM_SET_COMPARE`, removing any computational overhead from the main CPU.

3. Closed-Loop PI Control Loop (*Asservissement*)

- **Deterministic Execution:** The control loop executes inside the `HAL_TIM_PeriodElapsedCallback` triggered by **TIM4** at a fixed sample rate of **100 Hz** (every 10 ms).
- **Control Law:** Implements a Proportional-Integral (PI) discrete controller to track the target RPM:
$$u[n] = K_p \cdot e[n] + K_i \cdot \sum e[n]$$
- **Anti-Windup Protection:** To avoid integration saturation when the actuator hits its physical limits, the firmware features a software anti-windup clamping mechanism. The accumulated error is

bounded dynamically based on the maximum allowed threshold ($\$99 / K_i\$$). If the controller saturates against the safe minimum limit, the integral memory is dynamically recalculated to maintain loop stability.

4. "Diode Numérique" (Dynamic Braking Safety)

- A real-time dynamic saturation of the minimum PWM signal based on the kinetic energy of the flywheel.
- **The Formula:** The firmware calculates a variable lower bound for the PWM dynamically, utilizing the motor's empirical back-EMF constant:
$$\text{PWM}_{\text{min}} = K_e \cdot \text{Speed}_{\text{rpm}} - 10.0$$
- This software constraint prevents regenerative overcurrent from back-feeding and destroying the laboratory power supply during aggressive decelerations.

5. UART Communication Protocol

- Bidirectional control receiving commands via hardware interrupts (USART2).
- Parses incoming buffers for Open Loop (P), Closed Loop RPM target (V), Direction (S), and Emergency Brake (B).
- Telemetry data is streamed continuously in CSV format for external plotting: time_ms, rpm_AB, current_mA, full_rotations, rpm_I.

🔧 Hardware Architecture

- **Controller Board:** NUCLEO-F411RE (ARM Cortex-M4)
- **Power Driver:** H-Bridge / *Hacheur* driven by the 20 kHz PWM signal
- **Actuator/Sensors:** DC Motor coupled with an Optical/Magnetic Encoder and an analog current sensor

STM32 Pinout Map

PROF

Physical Pin	Microcontroller Function	Destination
PA8 (D7)	TIM1_CH1	PWM Power Signal (20 kHz)
PA9 (D8)	GPIO Output	Direction of Rotation
PA6 (D12)	GPIO Output	Emergency Brake Activation
PB4 / PB5	TIM3 (Encoder Mode TI12)	Encoder Channels A and B
PB6 / PB10	EXTI (External Interrupt)	Encoder Channel I (Index)
PA0 (A0)	ADC1_IN0	Current Sensor (Analog)

Développé dans le cadre du module d'Asservissement / Systèmes Commandés.

Professeur: M. Bourgeot